

Stigmergy and Hierarchical Learning for Routing Optimization in Multi-Domain Collaborative Satellite Networks

Yuanfeng Li[✉], Qi Zhang[✉], *Member, IEEE*, Haipeng Yao[✉], *Senior Member, IEEE*, Ran Gao, Xiangjun Xin[✉], and F. Richard Yu[✉], *Fellow, IEEE*

Abstract—The integration of Software-Defined Networking (SDN) and Artificial Intelligence (AI) presents promising opportunities for managing and optimizing LEO satellite network routing. However, as the scale and coverage of satellite networks continue to expand, challenges are posed to both centralized and distributed architectures in terms of managing network information and coping with routing complexity. To overcome these challenges, leveraging distributed SDN technology, a stigmergy multi-agent hierarchical deep reinforcement learning routing algorithm is proposed in multi-domain collaborative satellite networks. A pheromone-based mechanism is incorporated to facilitate collaboration during independent training, and hierarchical control is employed to decouple the complexity of cross-domain routing decisions. Simulation results demonstrate that our proposed algorithm exhibits good scalability and performance in large-scale satellite networks.

Index Terms—Low Earth orbit (LEO) satellite networks, distributed AI routing, stigmergy learning, hierarchical reinforcement learning.

I. INTRODUCTION

WITH the ongoing advancements in 5G technology, satellite networks are increasingly recognized as a critical element of future communication infrastructures [1], [2], [3]. This could significantly expedite the growth of the diverse Internet of Things (IoT), including smart wearables, industrial IoT, and other connected devices, all benefiting from enhanced communication capabilities [4], [5]. The recent launch of new low Earth orbit (LEO) satellite internet projects by innovative

companies such as SpaceX and OneWeb has reignited interest in the promising vision of seamless global communication through LEO satellites [6]. As LEO satellite networks become more integral to the communication infrastructure, routing optimization emerges as a fundamental challenge in ensuring their efficiency and effectiveness [7]. The dynamic nature of LEO satellite networks leads to the constant change in their topological connectivity over time, necessitating frequent maintenance and updates to the routing tables of each node [8]. As the scale of LEO satellite networks expands, obtaining real-time network status information becomes a challenge [9]. Excessive flooding transmissions result in substantial overhead for network state monitoring. Moreover, the increased scale of the network adds complexity to routing algorithms, posing challenges to their scalability and real-time performance.

Existing solutions to address the routing problem in LEO satellite networks can generally be categorized into three approaches: centralized, distributed, and hybrid routing methods.

Centralized routing methods rely on an SDN controller acting as a central authority, separating control and data planes. The controller calculates the optimal routing path using an incorporated algorithm and disseminates decisions to satellite nodes for efficient data packet transmission along optimal routes. For example, Li et al. [10] proposed an elastic resource allocation algorithm based on dispersion degree (ERA-DD) for satellite elastic optical networks with a centralized SDN architecture, addressing immediate reservation (IR) and advanced reservation (AR) services. Centralized routing can achieve fine-grained routing decisions due to its global network information access. However, the management complexities and payload overhead involved in forwarding *flow_mod* messages from the SDN controller to the corresponding nodes become a challenge as the network scale expands.

Distributed routing methods involve each satellite node independently calculating its routing decisions based on local information and exchanging data with its neighbors. There is no central control entity, and the routing process is performed by individual nodes [9], [11]. For instance, Chen et al. [12] proposed a distributed routing algorithm for large-scale constellation networks, focusing on gateway assignment and congestion avoidance. Distributed routing algorithms reduce network load and management pressure through limited flooding between neighboring nodes. However, relying on local

Manuscript received 4 June 2023; revised 13 November 2023; accepted 15 December 2023. Date of publication 18 March 2024; date of current version 9 May 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB2902502. (Corresponding authors: Qi Zhang; Haipeng Yao.)

Yuanfeng Li, Qi Zhang, and Xiangjun Xin are with the School of Electronic Engineering, the Beijing Key Laboratory of Space-Ground Interconnection and Convergence, and the State Key Laboratory of Information Photonics and Optical Communications, Beijing University of Posts and Telecommunications (BUPT), Beijing 100876, China (e-mail: liyuanfeng747@gmail.com; zhangqi@bupt.edu.cn; xjxin@bupt.edu.cn).

Haipeng Yao is with the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications (BUPT), Beijing 100876, China (e-mail: yaohaipeng@bupt.edu.cn).

Ran Gao is with the School of Information and Electronics, Beijing Institution of Technology, Beijing 100081, China (e-mail: gaoran@bit.edu.cn).

F. Richard Yu is with the School of Information Technology, Carleton University, Ottawa, ON K1S 5B6, Canada (e-mail: richard.yu@carleton.ca).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JSAC.2024.3365878>.

Digital Object Identifier 10.1109/JSAC.2024.3365878

information may cause routing decisions to deviate from the global optimum, necessitating a trade-off between network state acquisition and management granularity.

Hybrid routing, a distributed SDN routing method, adeptly combines centralized and distributed approaches, leveraging their advantages while mitigating their shortcomings. This method divides the network into several subdomains, each managed by a local control entity, and offers the potential to decouple satellite network scale and routing computation complexity, presenting promising prospects. Ravuri et al. [13] conducted a comparative analysis of different routing approaches' effects on the scalability of flat distributed SDN architectures, utilizing the Open Network Operating System (ONOS) as the evaluation architecture. Despite this, current research in this area is limited, and several challenges remain to be addressed. The primary challenge is routing collaboration between different domains, where domain SDN controllers must account for neighboring domains' forwarding decisions, identify the optimal neighboring domain for forwarding, and choose the appropriate inter-domain entry point. Furthermore, adjacent domains need to exchange network state information to inform routing decision-making for other domain controllers. Efficiently executing this information exchange presents another challenge that warrants resolution. Consequently, investigating hybrid routing methods in distributed SDN carries substantial value.

Deep reinforcement learning (DRL) is a promising technique that combines the strengths of deep learning with reinforcement learning, enabling an agent to learn optimal decisions through trial and error. DRL has been applied to various domains, including routing optimization, due to its ability to handle complex problems and adapt to changing environments. In this work, we propose a novel hybrid routing solution for LEO satellite networks, leveraging the power of DRL to address the challenges of scalability and real-time performance. The primary contributions of this study are as follows:

1) Scalable Routing via Multi-Domain Collaborative Routing: Routing solutions in satellite networks typically employ either completely centralized or distributed methods. While centralized methods can gather extensive network information, they stress the network and challenge the SDN controller's computational capacity. Distributed methods, on the other hand, may lack sufficient network information for optimal routing despite reducing network load. To overcome these issues, we propose a novel multi-domain collaborative routing approach. This approach is built upon a hybrid routing method that combines centralized and distributed methods, enhancing scalability.

2) Pheromone-Based Reward Mechanism for Agent Collaboration: Existing strategies for enhancing collaboration during distributed agent training often rely on sharing each agent's observation spaces. However, these methods present issues, such as high communication overhead and increased computational complexity. The requirement for each agent to access others' observation spaces can also be impractical or undesirable in real-world scenarios. To address these challenges, we introduce a pheromone-based reward mechanism, replacing

traditional multi-agent trajectory concatenation and enabling stigmergy.

3) Efficient Hierarchical DRL-Based Routing Approach: Deep reinforcement learning (DRL) methods for cross-domain routing in satellite networks often confront action complexity and sub-optimal decision-making. The crux of the problem lies in agents failing to identify the underlying sub-goal, resulting in an overly complex action space and challenging training. To tackle this, we propose an efficient hierarchical DRL-based routing approach. This method simplifies the process into two steps: sub-goal node selection and intra-domain routing optimization, facilitating easier training. Our approach is demonstrated in the Stigmergy Multi-Agent Hierarchical Deep Reinforcement Learning routing algorithm.

The rest of this paper is organized as follows. In Section II some related works will be introduced. Section III presents our system model and problem formulation. Section IV will introduce our proposed stigmergy multi-agent hierarchical reinforcement learning routing algorithm, including the algorithm framework, the configuration of agents, and algorithm training execution. Section V centers on the simulation results and related analysis. Finally, Section VI summarizes the main conclusions of this paper.

II. RELATED WORK

In this section, we will briefly discuss the related work on satellite network SDN control architectures, satellite network routing algorithms, and various types of AI routing methods.

A. Satellite Network Routing Optimization Algorithms

In terms of satellite network routing optimization algorithms, the most common approach is the Open Shortest Path First (OSPF) routing protocol based on the shortest path algorithm. However, in dynamic satellite networks, the OSPF protocol requires updating routing tables according to real-time network status, resulting in slow route convergence and significant network overhead. To address these challenges, researchers have proposed dynamic shortest path algorithms, such as the orbit prediction shortest path first routing [14], specifically designed for LEO satellite networks. This approach leverages constellation regularity for periodic route calculations, enabling instantaneous routing table generation, and effectively reducing communication overhead and route convergence time. Furthermore, due to the complexity and dynamic nature of satellite network environments, causing inter-satellite link instability, several heuristic routing algorithms based on link attributes have been proposed and effectively avoid these unstable paths [15].

In satellite optical network routing and wavelength assignment, meta-heuristic algorithms like ant colony have been extensively studied by scholars. However, while ant colony routing optimization algorithms have shown promising results in optimization, they face real-time challenges in terms of convergence speed and time complexity. Some research efforts have been made to improve these aspects. The Fast Convergence RWA based on Ant Colony Algorithm (ACO-FC RWA)

is proposed [16] to improve convergence speed. The ant colony optimization with the small window strategy (ACO-SWS) aims to reduce hop count and iterations by limiting the convergence area [17].

Currently, AI-based satellite network routing optimization mainly relies on deep reinforcement learning algorithms, which can be broadly divided into two categories: centralized routing and distributed routing (with each node as an agent). Centralized deep reinforcement learning is suitable for small-scale satellite networks, and by improving deep reinforcement learning mechanisms (e.g. Collaborative Q) [18], it selects precomputed optimal paths. On the other hand, distributed deep reinforcement learning algorithms are based on improvements to the Q-routing foundation [19], [20]. For the first method, there is usually an issue of insufficient scalability. As for the second method, it has some shortcomings in terms of overall routing optimization effectiveness and algorithm efficiency.

B. Related AI Techniques for Collaborative Routing Computation

1) *Stigmergy Learning*: Stigmergy is a term originating from the field of biology, specifically in the study of social insects like ants and termites. It refers to a mechanism of indirect communication and coordination among agents or individuals through modifications made to their environment [21]. These modifications serve as cues or stimuli, guiding the behavior of other agents that encounter them later. Stigmergy plays a crucial role in self-organization and swarm intelligence, enabling the emergence of complex, coordinated behavior without the need for centralized control or direct communication between agents.

Xu et al. [22] proposed a swarm deep reinforcement learning algorithm that utilizes ant colony pheromones as a collaborative reward factor for swarm intelligence. In their research, swarm agents achieved efficient collaboration with over 100 agents through the pheromone mechanism. Similarly, Bol et al. [23] proposed a swarm intelligence algorithm that leverages pheromones to guide agent actions. This approach also achieved better results than baseline algorithms like MAPPO in a public test environment with 75 agents.

2) *Hierarchical Deep Reinforcement Learning*: Hierarchical Deep Reinforcement Learning (HDRL) is an advanced approach in the field of Reinforcement Learning (RL) that aims to address the challenges of learning complex tasks with long time horizons and high-dimensional state and action spaces [24]. HDRL builds upon traditional deep reinforcement learning by incorporating hierarchical structures, which enable more efficient learning, better exploration, and improved generalization. The key idea behind HDRL is to decompose a complex task into a hierarchy of simpler subtasks, with each subtask being managed by a separate agent or policy. This hierarchical structure facilitates learning by allowing agents to focus on a specific subtask, reducing the overall complexity and enabling more efficient exploration and exploitation of the environment.

One popular method in HDRL is the options framework, where high-level policies, known as options, are responsible

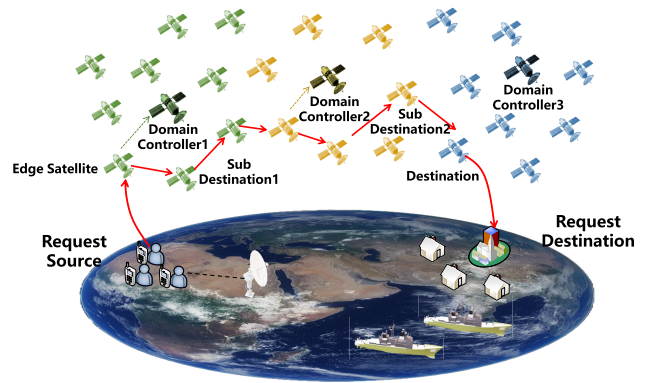


Fig. 1. Satellite Network Cross-Domain Routing Scenario.

for selecting and executing low-level policies to complete sub-tasks [25]. Temporal Abstraction is another critical component of HDRL, allowing agents to make decisions at different time scales and thereby accelerating the learning process [26].

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we present the system model and problem formulation of satellite network routing optimization. To more clearly illustrate the proposed architecture, a cross-domain routing scenario for a satellite network based on a distributed SDN architecture will be presented first. As illustrated in Fig. 1, a ground user's communication request is transmitted to the satellite network via an edge satellite (the nodes have connections to satellite nodes in other domains). Each satellite domain has a satellite domain controller based on an SDN controller. Upon receiving a routing request from an edge satellite node, the domain controller inserts the routing decision (relative node's SID) into the segment routing list. The request then undergoes two cross-domain forwarding instances before reaching its intended destination. It should be noted that, in this paper, we did not delve into the specific deployment strategies for controllers and their consequent impact on the network. The issue of SDN controller placement in satellite networks has already been explored in previous studies in [27] and [28].

A. System Model

Consider the satellite network consisting of a set of satellite domain $D = \{1, 2, \dots, d\}$. In addition, in each domain d , there is a set of satellite nodes $N^d = \{1, 2, \dots, n\}$. Similarly, in domain d there is a set of links $E^d = \{e_{i,j}^d, i, j \in N^d, i \neq j\}$, a set of cross domain links $E^{d,d'} = \{e_{i,j}^{d,d'}, i \in N^d, j \in N^{d'}, i \neq j\}$. Besides, a set of bandwidth resources is defined as $B^d = \{b_{i,j}^d, i, j \in N^d, i \neq j\}$ in domain d , and a set of cross-domain bandwidth resources $B^{d,d'} = \{b_{i,j}^{d,d'}, i \in N^d, j \in N^{d'}, i \neq j\}$.

In our article, we focus on the task of routing path optimization for satellite networks. Firstly, an essential aspect is the quality metric of a routing path, referred to as the routing cost metric, u_n . This metric consists of two components: link utilization, u_n^{util} , and delay, u_n^{delay} . Here, n represents the

n th routing request. These two components together form the overall cost metric for a routing path, representing both the efficiency of network resource usage and the speed of data transmission.

The link utilization metric u_n^{utl} can be further described as follows:

$$u_n^{utl} = \sum_{d,d' \in D} \sum_{i,j \in p_n(\text{sour}, \text{des})} \left(\frac{b_{i,j}^{used,d}}{b_{i,j}^{all,d}} \right) + \left(\frac{b_{i,j}^{used,d,d'}}{b_{i,j}^{all,d,d'}} \right), \quad (1)$$

where $p_n(\text{sour}, \text{des})$ denotes the set of all links on the path from source node *sour* to destination node *des* for routing request n , and D is the set of all domains. $p_n(\text{sour}, \text{des})$ could be represented as $[(\text{sour}, \text{node1}), (\text{node1}, \text{node2}), \dots, (\text{nodeK}, \text{des})]$. The used bandwidth resources and total bandwidth resources of the link between nodes i and j in domain d are represented by $b_{i,j}^{used,d}$ and $b_{i,j}^{all,d}$, respectively. Similarly, for a link between a node i in domain d and a node j in another domain d' , the used and total bandwidth resources are represented by $b_{i,j}^{used,d,d'}$ and $b_{i,j}^{all,d,d'}$, respectively. These represent the bandwidth resources that are used and available, respectively, on the cross-domain link from node i in domain d to node j in domain d' .

As demonstrated in Equation (2), we compute the delay metric u_n^{delay} by summing the delay of all links on the routing path p across all domains, which can be denoted as:

$$u_n^{delay} = \sum_{d,d' \in D} \sum_{i,j \in p_n(\text{sour}, \text{des})} delay_{i,j}^d + delay_{i,j}^{d,d'}. \quad (2)$$

Here, u_n^{delay} represents the delay metric computed by aggregating the delay of all links on the routing path p across all domains.

Furthermore, each hop routing delay can be further expressed as:

$$delay_{i,j} = delay_{i,j}^{proc} + delay_{i,j}^{trans}, \quad (3)$$

here, we define the per-hop delay $delay_{i,j}$, consisting of two parts: processing delay $delay_{i,j}^{proc}$ and transmission delay $delay_{i,j}^{trans}$.

Due to the instability of inter-satellite links (ISLs) caused by various factors, it is necessary to perform mathematical modeling of their link attributes from multiple perspectives.

Bit Error Rate Analysis: During satellite network transmission, a high BER can result in unrecognized data and cause routing service request failures. Therefore, it is essential to maintain the BER within a reasonable range. For inter-satellite links, the BER should be less than 1×10^{-6} . The relationship equation between the Doppler shift and BER can be expressed as:

$$BER = \frac{1}{2}Q(\sqrt{G_1(\Delta\lambda)}) + \frac{1}{4}Q(\sqrt{G_1(\Delta\lambda)}(1 + \frac{2}{G_2(\Delta\lambda)})) + \frac{1}{4}Q(\sqrt{G_1(\Delta\lambda)}(1 - \frac{2}{G_2(\Delta\lambda)})), \quad (4)$$

where $G_1(\cdot)$ and $G_2(\cdot)$ represent the mapping relationship between two frequency shift values and BER as a function. Reference [29] indicates that the Doppler shift value is positively correlated with BER, and the threshold $\Delta\lambda_{th}$ was calculated as $3.22 \times 10^{-11}m$.

ISL Duration Analysis: Due to the dynamic characteristic of the satellite constellation, not only links exceeding the BER threshold will fail but also inter-orbit links will fail due to phase difference changes of neighboring satellites (phase difference leads to relative position changes). In the Walker constellation, which consists of LEO satellites, the ISL is disconnected after the satellite enters the vicinity of the polar region. The duration of ISL is expressed as:

$$l_{i,j}(t) = \begin{cases} t_b - t; & 0 \leq t \leq t_b \\ 0; & t_b \leq t \leq t_b + T_d \\ T/2 - t + t_b; & t_b + T_d \leq t \leq T/2, \end{cases} \quad (5)$$

where $l_{i,j}(t)$ represents the maintenance time function of link $e_{i,j}$ in domain d at moment t . t_b represents the moment of link disconnection within one satellite movement cycle T . ω_β is the phase angle between the polar border and the equator line. T_d represents the duration of the link disconnection and it can be further expressed as:

$$T_d = \frac{\pi - 2\omega_\beta + \Delta\omega_f}{v_s}, \quad (6)$$

where $\Delta\omega_f$ denotes the phase difference between adjacent satellites in the Walker constellation, it can be expressed as follows:

$$\Delta\omega_f = \frac{2\pi \cdot F}{R \cdot C}, \quad (7)$$

where F is the phase factor, R is the number of constellation orbits, C is the number of satellites per orbit.

B. Problem Formulation

After considering the above factors affecting routing transmission in satellite networks, the following optimization objectives and constraints can be established:

$$\min_{p_n(\text{sour}, \text{des})} u_n^{utl}, \quad (8a)$$

$$\min_{p_n(\text{sour}, \text{des})} u_n^{delay}. \quad (8b)$$

Subject to the following constraints:

$$C1: \sum_{d,d' \in D} \sum_{i,j \in p_n(\text{sour}, \text{des})} y_{i,j}^{des} + delay_{i,j}^{d,d'} \leq QoS_n^{delay}, \quad (9a)$$

$$C2: b_{i,j}^{all,d} - b_{i,j}^{used,d}(t) \geq QoS_n^{band}, \quad b_{i,j}^{all,d,d'} - b_{i,j}^{used,d,d'}(t) \geq QoS_n^{band}, \quad \forall i, j \in p_n(\text{sour}, \text{des}), \quad (9b)$$

$$C3: l_{i,j}^d(t) \geq QoS_n^{dur}, \quad l_{i,j}^{d,d'}(t) \geq QoS_n^{dur}, \quad \forall i, j \in p_n(\text{sour}, \text{des}), \quad (9c)$$

$$C4: |\Delta\lambda_{i,j}^d(t)| \leq \Delta\lambda_{th}, \quad |\Delta\lambda_{i,j}^{d,d'}(t)| \leq \Delta\lambda_{th}, \quad \forall i, j \in p_n(\text{sour}, \text{des}). \quad (9d)$$

In the optimization objective outlined in Equation (8), our aim is to fulfill each individual routing request. This is achieved by selecting a path $p_n(\text{sour}, \text{des})$ that minimizes link utilization while simultaneously ensuring the lowest possible delay.

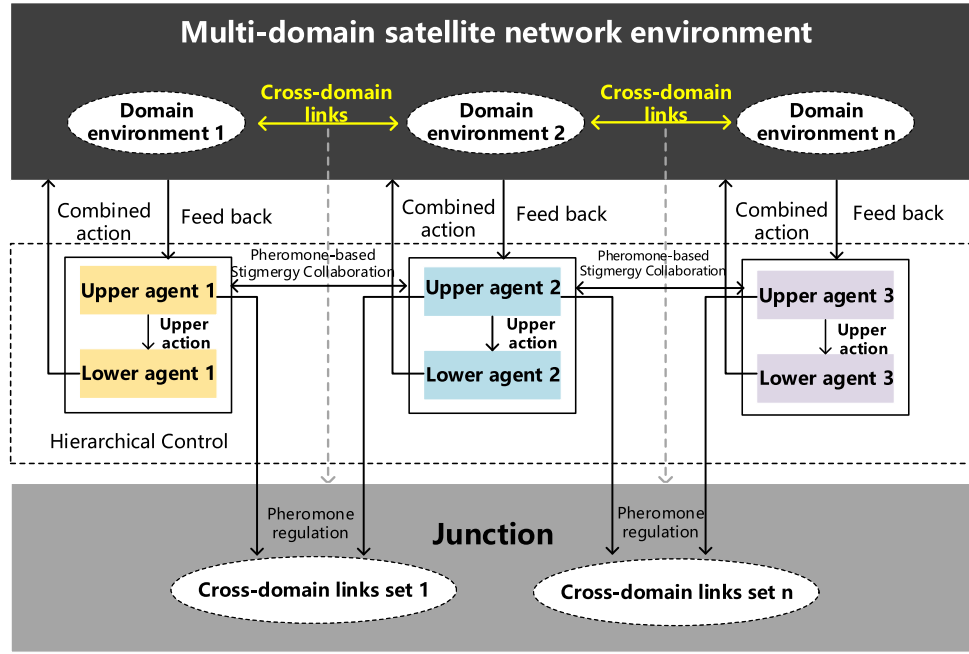


Fig. 2. Algorithm Framework.

Constraint C1 stipulates that the total delay across all links within the path $p_n(\text{sour}, \text{des})$ for a given independent routing request n should not exceed the Quality of Service (QoS) delay requirement threshold, QoS_n^{delay} , for that specific request n .

Constraint C2 asserts that the residual bandwidth resources on the chosen routing path must be sufficient to satisfy the QoS bandwidth requirements, QoS_n^{band} , for the routing request n .

Constraint C3 mandates that all links within the selected routing path need to be maintained for a duration exceeding the QoS hold time demand, QoS_n^{dur} , for the routing request n .

Finally, Constraint C4 indicates that every link on the routing path should have a Doppler shift, $\Delta\lambda_{i,j}^d(t)$ and $\Delta\lambda_{i,j}^{d'}(t)$, smaller than the acceptable Doppler shift threshold, $\Delta\lambda_{th}$.

IV. STIGMERGY MULTI-AGENT HIERARCHICAL REINFORCEMENT LEARNING ROUTING ALGORITHM

In this section, we propose the stigmergy multi-agent hierarchical reinforcement routing algorithm. We will initially introduce an algorithm framework. Subsequently, we will provide a detailed introduction to the design and related mechanisms of the upper and lower agents. Finally, this section will present a detailed pseudo-code for the training of the agents.

A. Algorithm Framework

As shown in Fig. 2, it describes the framework of our proposed algorithm.

Environment: The multi-domain satellite network environment is fundamental to training agents. There are cross-domain links between some domain satellite networks for connectivity, defined as junctions. In the multi-agent system (MAS), each agent can only observe the network state to which it belongs.

After receiving a combined action from the agent, the environment gives the agent feedback, which includes the next state and the independent joint reward.

Agent: The agents part consists of upper agents and lower agents. The upper agents are responsible for cross-domain routing decisions. By adjusting the pheromones in the junctions, every two adjacent upper agents achieve collaborative cross-domain routing decisions among multiple intelligent agents (i.e., pheromone-based stigmergy collaboration). The lower agents are in charge of intra-domain routing optimization. They adopt Graph Neural Networks to conduct message passing for network status distribution, making intra-domain routing decisions.

Junction: We refer to cross-domain links that connect different domains as junctions. Junction is a shared part of different domains, so it can play a key role in collaboration with agents. The pheromones are distributed in the junction to give guidance to the upper agent. The rules for pheromones are determined by the state of the perimeter links of the cross-domain links, the results of the routing history.

B. Preliminaries

We consider a fully cooperative multi-agent task a decentralized partially observable Markov decision process (Dec-POMDPs) [30] that describes multi-agent routing decisions in the multi-domain satellite network. Dec-POMDPs can be formally defined by a tuple $\langle \mathcal{N}, \mathcal{S}, \mathcal{U}, P, r, \mathcal{Z}, O, n, \gamma \rangle$. In this article, $s \in \mathcal{S}$ denotes the multi-domain satellite network environment. $\mathcal{N}_u = \{1, \dots, N\}$ and $\mathcal{N}_l = \{1, \dots, N\}$ respectively denote a set of N upper and lower agents. n is the initial state distribution. At each time step t , all agents, upper agent $a_u \in \mathcal{N}_u$ or lower agent $a_l \in \mathcal{N}_l$ could simultaneously choose actions $\mathbf{u}_t = \{u_t^a\}_{a=1}^N \in \mathcal{U}^N$. Instead of being able to observe the full state, in the Dec-POMDPs, each agent

(i.e., upper or lower agent) could only obtain individual local observation $z_t^a \in \mathcal{Z}$, $z_t = \{z_t^a\}_{a=1}^N$ from the observation function $O(s_t, a)$. The next state $s_{t+1} \sim P(s_t, u_t)$ is drawn from transition function P after executing the joint action u_t in state s_t . Then, each agent a obtains an independent reward $r_t^a = r(z_t^a, u_t^a)$.

C. Upper Agent Configurations

1) *Definition of Upper Agent:* For each upper agent, an independent optimal action-value function $Q^{*,a,U}(z^{a,U}, u^{a,U})$ can be defined as:

$$Q^{*,a,U}(z^{a,U}, u^{a,U}) = \mathbb{E}_{\pi^L(z^{a,U}, u^{a,U})} \left[r(z^{a,U}, u^{a,U}, u^{a,L}) + \gamma \max_{u^{a,U'}} Q^{*,a,U}(z^{a,U'}, u^{a,U'}, u^{a,L}) \right], \quad (10)$$

where z^a denotes the current state, $u^{a,U}$ represents the upper agent's action, and $u^{a,L}$ stands for the lower agent's action. π^L symbolizes the policy of the lower agent. The immediate reward obtained by taking the upper agent action $u^{a,U}$ and the lower agent action $u^{a,L}$ at state z^a is denoted by $r(z^a, u^{a,U}, u^{a,L})$. γ is the discount factor that weighs the importance of immediate and future rewards. $Q^{*,a,U}(z^{a,U}, u^{a,U})$ represents the maximum expected reward for taking the upper agent action $u^{a,U'}$ at the next state $z^{a,U'}$ under the current lower agent policy $\pi^L(z^{a,U'}, u^{a,U'})$.

Furthermore, we redefine the loss function $L_i^{a,U}(\theta_i^{a,U})$ for the upper agent a with parameters $\theta_i^{a,U}$ in the i -th iteration as follows:

$$L_i^{a,U}(\theta_i^{a,U}) = \mathbb{E} \left[\left(y_i^{a,U} - Q^{*,a,U}(z^{a,U}, u^{a,U}; \theta_i^{a,U}) \right)^2 \right]. \quad (11)$$

Equation (11) essentially represents a regression problem, where the objective is to minimize the error between the predicted action-value function $Q^{*,a,U}$ and the target value $y_i^{a,U}$, thereby optimizing the decision-making policy of the upper agent. $\theta_i^{a,U}$ represents the parameters of the neural network used to predict the Q-value function for the upper agent a .

By differentiating the loss function with respect to the weights, we obtain the following independent gradient:

$$\begin{aligned} & \nabla_{\theta_i^{a,U}} L_i^{a,U}(\theta_i^{a,U}) \\ &= \mathbb{E} \left[(r + \gamma \max_{u^{a,U'}} Q(z^{a,U'}, u^{a,U'}; \theta_{i-1}^{a,U}) - (z^{a,U}, u^{a,U}; \theta_i^{a,U})) \nabla_{\theta_i^{a,U}} Q(z^{a,U}, u^{a,U}; \theta_i^{a,U}) \right]. \end{aligned} \quad (12)$$

With the redefined formula in place, we will proceed to describe the state, action, and reward settings for the upper agents in accordance with the previously stated requirements.

State Definition: In this paper, we define the observation state for the upper agent, which can be denoted as:

$$z^{a,U} = (BW_{i,j}^{d,d'}, Dis_{i,j}^{d,d'}, Dop_{i,j}^{d,d'}, Phi_{i,j}^{d,d'}, Flag_{i,j}^{d,d'}, Hid), \quad (13)$$

where $BW_{i,j}^{d,d'}$ represents the bandwidth margin of each link, $Dis_{i,j}^{d,d'}$ denotes the distance of each link, $Dop_{i,j}^{d,d'}$ represents the Doppler shift of each link, $Phi_{i,j}^{d,d'}$ signifies the pheromone intensity of the link, $Flag_{i,j}^{d,d'}$ indicates whether each link is marked by the domain routing table, and Hid represents information hiding. It refines the regional link usage of the satellite domain network and marks the source node of the route.

We select these features based on the following considerations:

1) By observing the bandwidth, distance, and Doppler shift of the cross-domain links, the upper agent can avoid selecting links that lack resources, do not comply with the Doppler constraints, or have high latency as inter-domain routing links. This helps to ensure the efficiency and reliability of cross-domain routing.

2) By observing the strength of the pheromone, the upper agent can obtain a brief overview of the network state of neighboring domains, and refer to the actual cross-domain link quality observed before making basic routing decisions. It helps to make routing decisions that better match the actual network conditions.

3) The flagged links allow the upper agent to observe which inter-domain link was actually chosen by its routing decision, which helps the upper agent to give feedback and adjust its decisions.

4) Hidden information enables the agent to estimate the difficulty of cross-domain routing brought by the cross-domain link to the lower agent, thereby selecting cross-domain links that are more suitable for intra-domain routing for transmission.

Action Definition: We define actions for the upper agent, which can be expressed as:

$$u^{a,U} = (Flag_{i,j}^{d,d'}), \quad (14)$$

where $Flag_{i,j}^{d,d'}$ denotes alternative cross-domain links flagged according to the domain routing table. This gives the upper agent the ability to choose among these flagged links for cross-domain routing, providing a sub-destination for the lower agent.

Rewards Definition: The reward function for the upper agent can be expressed as:

$$r^{a,U}(z^a, u^{a,U}, u^{a,L}) = \alpha \cdot e^{(phi_{\min} - phi_{sel})} + \beta \cdot r^{hid}(z^{a,L}, u^{a,L} | z^a, u^{a,U}), \quad (15)$$

Here, phi_{sel} denotes the pheromone intensity of the selected link in the junction, and $phi_{\min}$ represents the minimum pheromone intensity of the flagged link.

The first part of the equation, $e^{(phi_{\min} - phi_{sel})}$, indicates that the reward for the selected cross-domain link increases as the pheromone intensity decreases. This is due to the properties of the exponential function.

Furthermore, the hidden reward (r^{hid}) for the lower agents also contributes to the rewards of the upper agents. This means that the upper agent needs to estimate whether the lower agent can successfully perform its task, as this directly impacts the reward for the upper agent.

The parameters α and β are weight parameters used to balance the influence of the pheromone intensity and the performance of the lower-level agents on the reward function. It should be noted that agents only receive rewards when cross-domain routing is successful. If cross-domain routing fails, the reward is a negative constant r .

2) *Pheromone-Based Stigmergy Collaboration*: For cross-domain links $e_{i,j}$, we define them as junctions. For each agent of domain d , it exists a set of cross-domain links $E^{d,d'} = \{e_{i,j}, i \in N^d, j \in N^{d'}, d \neq d'\}$. For each $e_{i,j} \in E^{d,d'}$, it has the pheromone intensity property $ph_{i,j}(t)$. In the satellite network, the pheromone intensity of each cross-domain link is a time variable. In this article, at the moment $t = 0$, the initial pheromone intensity $ph_{i,j}^{d,d'}(0)$ is 0. Further, we set up the pheromone intensity rule formula, which can be expressed as:

$$ph_{i,j}^{d,d'}(t + \Delta t) = \rho(t + \Delta t) \cdot [ph_{i,j}^{d,d'}(t) + \sum_n \tau_{(i,j),n}^{d,d'}(t + \Delta t)]. \quad (16)$$

As shown in equation (16), the rule of variation of the real-time pheromone intensity $ph_{i,j}^{d,d'}(t + \Delta t)$ in across the domain link $e_{i,j}$ is influenced by three parts. The first part is the pheromone intensity $ph_{i,j}^{d,d'}(t)$ from the previous Δt time interval. We set the initial pheromone intensity $ph_{i,j}^{d,d'}(0)$ to be 0 at time $t = 0$. The second part is a conditional function $\tau_{(i,j),n}^{d,d'}(t)$ that combines link states and routing traffic history to determine the increase in pheromones. As can be seen from the equation, the expression in the square brackets represents the sum of the pheromone intensity from the previous moment and the change in the conditional function for pheromones. The third part is the pheromone evaporation function $\rho(t)$. This function proportionally evaporates the pheromones at a certain period (i.e., $t = t_e$) to prevent pheromones' indefinite accumulation on the cross-domain link.

The pheromone evaporation function $\rho(t)$ is designed to avoid infinite accumulation of pheromones in certain cross-domain links during the lifetime of the satellite network. It can be further expressed as:

$$\rho(t) = \begin{cases} \alpha & t = t_e \\ 1 & \text{else,} \end{cases} \quad (17)$$

here t_e denotes the moment of pheromone evaporation and α is a constant less than 1. We set the pheromone to evaporate at a specific frequency.

The condition function $\sum_n \tau_{(i,j),n}^{d,d'}(t)$ is a set of decision variables which is the principle for determining how pheromones change. n denotes the n th situation to dynamically change the pheromone intensity. We consider pheromone changes in terms of both the link state of the links surrounding the cross-domain link and the results of the recent routing history. Here we make more detailed definitions:

$$\tau_{(i,j),1}^{d_m,d_n}(t + \Delta t) = \sum_{d'} \mathcal{R}^{d',d_m}(t + \Delta t) - \mathcal{R}^{d',d_m}(t) \quad (18a)$$

$$\tau_{(i,j),2}^{d_m,d_n}(t + \Delta t) = \mathcal{R}_{(i,j)}^{d_m,d_n}(t + \Delta t) - \mathcal{R}_{(i,j)}^{d_m,d_n}(t) \quad (18b)$$

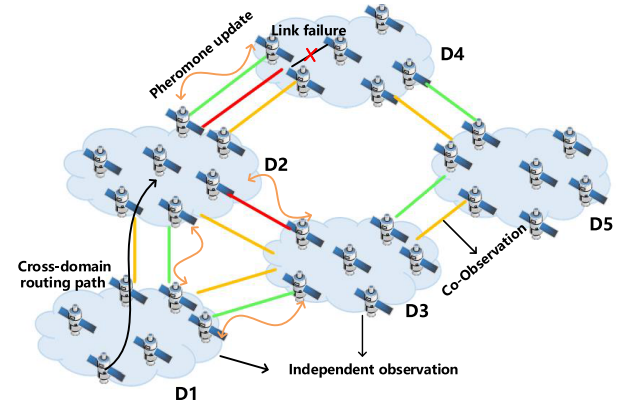


Fig. 3. Stigmergy collaborative process.

$$\tau_{(i,j),3}^{d_m,d_n}(t + \Delta t) = \begin{cases} \mu & \exists \left| \Delta \lambda_{i,j'}^{d_m}(t + \Delta t) \right| > \Delta \lambda_{th} \\ 0 & \text{else} \end{cases} \quad (18c)$$

$$\tau_{(i,j),4}^{d_m,d_n}(t + \Delta t) = \begin{cases} \omega & \exists l_{i,j'}^{d_m}(t + \Delta t) = 0 \\ 0 & \text{else.} \end{cases} \quad (18d)$$

Equations (18a) and (18b) illustrate how cross-domain routing history affects pheromone accumulation. $\mathcal{R}^{d',d_m}(t)$ denotes the routing traffic history passing through domain d_m at moment t from any adjacent domain d' . $\mathcal{R}_{(i,j)}^{d_m,d_n}(t)$ symbolizes the routing traffic history passing through the cross-domain link $e_{i,j}^{d_m,d_n}$ at the same moment. Equation (18a) shows that routing history from any adjacent domain d' through domain d_m results in pheromone accumulation on the cross-domain link $e_{i,j}^{d_m,d_n}$, as agents must consider adjacent domain link utilization during cross-domain routing decisions. Equation (18b) takes into account the effect of cross-domain routing history through cross-domain link $e_{i,j}^{d_m,d_n}$ on pheromone accumulation. It should be noted that the routing traffic history is represented as a scalar, incrementing by one with each routing request, thereby forming a traffic history.

Equations (18c) and (18d) demonstrate how adjacent links' states affect pheromone accumulation on the cross-domain link $e_{i,j}^{d_m,d_n}$. In (18c), $\Delta \lambda_{i,j'}^{d_m}$ represents the absolute Doppler shift of any neighboring link of the cross-domain link. If a Doppler shift of an adjacent link surpasses the threshold, it increases pheromone accumulation by μ units on the cross-domain link. Equation (18d) defines $l_{i,j'}^{d_m}$ as the maintenance time of any adjacent link, and if a link's maintenance time is zero, pheromone accumulation on the cross-domain link increases.

Fig. 3 depicts the stigmergy collaboration process in a macroscopic satellite network scenario with five domains, where adjacent domains connect via cross-domain links. Colors indicate pheromone intensity on cross-domain links (green, yellow, and red for low, medium, and high intensity, respectively). As demonstrated, a cross-domain request from D1 to D2 increases pheromone intensity on all cross-domain links with D1, D3, and D4, and further increases it on the selected routing path. A link failure in D4 leads to an increase

in pheromone intensity on the cross-domain link between D4 and D2.

D. Lower Agent Configurations

Deep reinforcement learning combined with graph neural networks has been shown to have some advantages in network routing optimization [31]. In this article, the lower agent focuses on intra-domain routing optimization (i.e., considering delay and load balancing while also avoiding failed links), so we implemented the GNN+DRL architecture. We adopted the message passing neural network, a well-known type of GNNs that is the graph neural network in our lower agent.

1) *The Architecture of Graph Neural Networks*: Graph Neural Networks (GNNs) extend the concepts of Deep Learning from data defined on regular grid structures (like images, audio, and texts) to irregular structures, particularly graph data. The main idea behind GNNs is to learn a function that can map the graph's input features to output features at nodes or links, or the entire graph. They perform this by leveraging the graph structure and applying recursive neighborhood aggregation or message-passing. Essentially, each node aggregates the features of its neighboring nodes and its own to compute a new feature, capturing the local structure around the node. This process is recursively repeated, enabling GNNs to capture representations of the graph.

In GNN, each link or node has its hidden state. The hidden state (also often referred to as the hidden representation or hidden feature) of a node (or a link) in a graph is a vector that captures useful information about the node's (or link) local neighborhood. This information is processed and updated over several iterations (or "layers") in the GNN, with each iteration expanding the scope of the neighborhood from which the node draws information. As shown in Fig. (4), the process of message passing between a link $e_{1,2}$ in the graph and its neighboring links $e_{2,3}$ and $e_{2,4}$ is illustrated. The message passing in the graph neural network consists of two main parts: the aggregation of messages from neighboring nodes, and the updating of the node representations based on the aggregated messages.

First, a link $e_{i,j}$ receives messages from its adjacent links. The message aggregation can be expressed as:

$$M_{e_{i,j}}^{t+1} = \sum_{e_{x,y} \in N(e_{i,j})} m(h_{e_{i,j}}^t, h_{e_{x,y}}^t), \quad (19)$$

where $N(e_{i,j})$ is the set of adjacent links to $e_{i,j}$, $h_{e_{i,j}}^t$ and $h_{e_{x,y}}^t$ are the representations of links $e_{i,j}$ and $e_{x,y}$ at time step t , and $m(h_{e_{i,j}}^t, h_{e_{x,y}}^t)$ is the message function that converts the representation of link $e_{x,y}$ to the message $M_{e_{i,j}}^{t+1}$ received by link $e_{i,j}$. Then, the representation $M_{e_{i,j}}^{t+1}$ of link $e_{i,j}$ is updated by aggregating the messages from all links $e_{x,y}$ in $N(e_{i,j})$.

Second, a link $e_{i,j}$ updates its own representation based on the next time step $M_{e_{i,j}}^{t+1}$ aggregated messages. The update function of the link representations can be expressed as:

$$h_{e_{i,j}}^{t+1} = t(h_{e_{i,j}}^t, M_{e_{i,j}}^{t+1}), \quad (20)$$

Here, $t(h_{e_{i,j}}^t, M_{e_{i,j}}^{t+1})$ is the update function that transforms the current representation $h_{e_{i,j}}^t$ of link $e_{i,j}$ and all the messages

$M_{e_{i,j}}^{t+1}$ it has received into a new representation $h_{e_{i,j}}^{t+1}$. In Graph Neural Networks (GNNs), the message function $m(\cdot)$ and the update function $t(\cdot)$ are typically modeled using neural networks. Through the process of training, the parameters of these neural networks are continuously optimized, thereby enhancing the effectiveness of these functions and improving the overall performance of the GNN.

2) *Definition of Lower Agent*: In this article, we have re-designed the optimal action-value function for the lower agents, as shown in Equation (21). The optimal action-value function for the lower agents can be expressed as:

$$\begin{aligned} Q^{*a,L}(z^{a,L}, u^{a,L} | z^a, u^{a,H}) \\ = \mathbb{E}[r^{hid}(z^{a,L}, u^{a,L}) \\ + \gamma \max_{u^{a,L'}} Q^*(z^{a,L'}, u^{a,L'} | z^a, u^{a,H})], \end{aligned} \quad (21)$$

where $Q^{*a,L}(z^{a,L}, u^{a,L} | z^a, u^{a,H})$ represents the lower agent's optimal action-value function, considering the current observation $z^{a,L}$, the lower agent's action $u^{a,L}$, the upper agent's observation z^a , and the upper agent's action $u^{a,H}$. The reward function $r^{hid}(z^{a,L}, u^{a,L})$ accounts for the reward obtained by the lower agent for taking action $u^{a,L}$ at the current state $z^{a,L}$. The discount factor γ balances the importance of immediate and future rewards.

Having redefined the formula, in particular, the following will describe the state, action and reward settings of the lower agents.

State Definition: The observation state for the lower agent can be denoted as:

$$z^{a,L} = (BW_{i,j}^d, Dis_{i,j}^d, Dop_{i,j}^d, Flag_{i,j}^d), \quad (22)$$

here, $BW_{i,j}^d$ represents the bandwidth margin of each link within the satellite domain d , $Dis_{i,j}^d$ denotes the distance of each link, $Dop_{i,j}^d$ signifies the Doppler shift of each link, and $Flag_{i,j}^d$ indicates the marking of multiple candidate routing paths using one-hot encoding on link states. Despite having fewer state features than upper agents, lower agents exhibit a larger state dimensionality due to the significantly higher number of intra-domain links compared to inter-domain links, leading to increased optimization challenges.

We select these features based on the following considerations:

By assessing the bandwidth, distance, and Doppler shift of the internal links within the satellite domain, the lower agent can sidestep the selection of links that are resource-deficient, incompatible with Doppler constraints, or characterized by high latency for intra-domain routing. This strategy contributes to the enhancement of the efficiency and reliability of routing within the domain. The role of $Flag_{i,j}^d$ is that it allows the lower agent to know which links have been chosen as routing paths for each of its routing actions.

Action Definition: The actions for the lower agent can be expressed as:

$$u^{a,L} = (Path_k), \quad (23)$$

here, $Path_k$ denotes the k -th precomputed routing path selected by the lower agent between each pair of nodes within every domain.

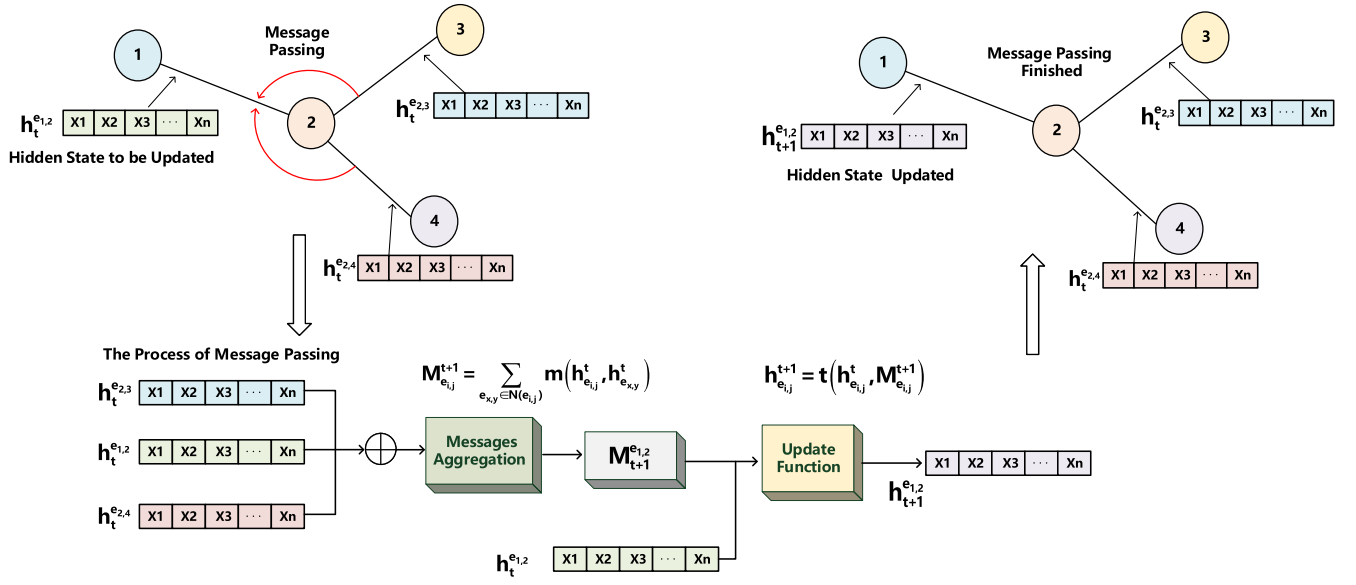


Fig. 4. The process of message passing of GNN.

Rewards Definition: Lower agents receive rewards from upper agents based on sub-goal completion, a process known as reward hiding. The reward function can be expressed as:

$$r^{hid}(z^{a,L}, u^{a,L} | z^a, u^{a,H}) = \lambda \cdot e^{BW_{sel} - BW_{max}} + \gamma \cdot e^{-delay}. \quad (24)$$

In Equation (24), the reward r^{hid} for the lower agent is decided by two factors. The first factor is the exponential of the difference between the maximum bandwidth BW_{max} in the domain and the average bandwidth margin of the desired path BW_{sel} . The second factor includes the delay of the selected path, represented in a negative exponential form e^{-delay} .

The weighting parameters, λ and γ , are used to balance the contribution of these two factors to the overall reward. By changing these weights, the influence of bandwidth margin and delay time on the reward function can be regulated.

E. Algorithm Training Execution

In this section, we have presented pseudocode for three algorithms: the message-passing algorithm, the pheromone-update algorithm, and the stigmergy hierarchical multi-agent deep reinforcement learning routing algorithm. Algorithm 1 and Algorithm 2 are utilized by Algorithm 3.

In Algorithm 1, the Message Passing Algorithm is presented. The algorithm takes as input an observation $z^{a,L}$ and a predefined maximum number of iterations T . The output of the algorithm is the Q-value Q and the final hidden states $h_{e_{i,j}}^T$. Initially, the functions $m(\cdot)$, $t(\cdot)$, and $read(\cdot)$ are defined. For all $z_{e_{i,j}}$ in $z^{a,L}$, padding is applied to obtain $h_{e_{i,j}}^0$. Then, for each iteration from 1 to T , the algorithm computes messages using the message function $m(\cdot)$ and aggregates messages from neighboring elements $e_{x,y}$ in $N(e_{i,j})$. After computing the messages, the hidden states are updated using the transformation function $t(\cdot)$. Finally, the algorithm returns the final hidden states $h_{e_{i,j}}^T$.

Algorithm 1 Message Passing Algorithm

Require: Input: observation $z^{a,L}$, Maximum Iterations T

Ensure: Output: final hidden states $h_{e_{i,j}}^T$

- 1: Initialize function $m(\cdot)$, $t(\cdot)$
- 2: **for all** $z_{e_{i,j}} \in z^{a,L}$ **do**
- 3: Padding $z_{e_{i,j}}$ to $h_{e_{i,j}}^0$
- 4: **for** $t = 1$ to T **do**
- 5: Compute message
 $M_{e_{i,j}}^{t+1} = \sum_{e_{x,y} \in N(e_{i,j})} m(h_{e_{x,y}}^t, h_{e_{i,j}}^t)$
- 6: Update hidden state $h_{e_{i,j}}^{t+1} = t(h_{e_{i,j}}^t, M_{e_{i,j}}^{t+1})$
- 7: **end for**
- 8: **end for**
- 9: **return** $h_{e_{i,j}}^T$

Algorithm 2 Pheromone Update

Require: upper observation $z^{a,H}$, pheromone distribution $Ph_{i,j}$, routing history $\mathcal{R}$, evaporation rate ρ , time t

Ensure: updated pheromone distribution $Ph_{i,j}$

- 1: Initialize the condition functions $\tau_{(i,j),n}^{d,d'}$
- 2: **for each** routing in $\mathcal{R}$ **do**
- 3: Update $\tau_{(i,j),1}^{d,d'}$ according to equation (18a)
- 4: Update $\tau_{(i,j),2}^{d,d'}$ according to equation (18b)
- 5: **end for**
- 6: **for each** link state $z_{i,j}^{a,H}$ in $z^{a,H}$ **do**
- 7: Update $\tau_{(i,j),3}^{d,d'}$ according to equation (18c)
- 8: Update $\tau_{(i,j),4}^{d,d'}$ according to equation (18d)
- 9: **end for**
- 10: Sum all condition function $\sum_n \tau_{(i,j),n}^{d,d'}$
- 11: Update $Ph_{i,j}$ according to equation (16)

In Algorithm 2, we introduce the Pheromone Update algorithm. This algorithm requires an upper observation $z^{a,H}$,

a pheromone distribution $Ph_{i,j}$, routing history $\mathcal{R}$, evaporation rate ρ , and time t as input parameters. It yields the updated pheromone distribution $Ph_{i,j}$ as output. Initially, the algorithm defines the condition functions denoted as $\tau_{(i,j),n}^{d,d'}$. Subsequently, for each routing instance within the routing history $\mathcal{R}$, updates are made to $\tau_{(i,j),1}^{d,d'}$ and $\tau_{(i,j),2}^{d,d'}$ in accordance with equations (18a) and (18b), respectively. Following this, for each link state $z_{i,j}^{a,H}$ present in the upper observation $z^{a,H}$, the algorithm proceeds to update $\tau_{(i,j),3}^{d,d'}$ and $\tau_{(i,j),4}^{d,d'}$ as per equations (18c) and (18d). Once all updates are performed, the algorithm calculates the summation of all the condition functions, represented as $\sum_n \tau_{(i,j),n}^{d,d'}$. Ultimately, the pheromone distribution $Ph_{i,j}$ is updated according to equation (16), and the revised pheromone distribution is returned.

In Algorithm 3, we present the Stigmergy Multi-Agent Hierarchical Deep Reinforcement Learning Routing Algorithm. The algorithm takes as input the environment Z , the number of upper agents a^U , the number of lower agents a^L , the number of training episodes M , discount factor γ , and target network update frequency T . The output consists of the trained Q-networks $Q^{a,U}$ and $Q^{a,L}$. The algorithm starts by initializing the upper and lower Q-networks, as well as their corresponding target Q-networks with the same initial weights. For each training episode, the environment is reset, and the initial states for both upper and lower agents are obtained. For each upper agent, the pheromone is updated using Algorithm 2. An action $u^{a,U}$ is selected using an ϵ -greedy strategy based on the upper Q-network $Q^{a,U}$. The action is executed, and a subgoal is provided to the lower agents. For each lower agent, message passing is performed using Algorithm 1. An action $u^{a,L}$ is selected using an ϵ -greedy strategy based on the lower Q-network $Q^{a,L}$. The action is executed, and the agent observes the reward hiding r^{hid} and the next state $z^{a,L'}$. Experiences are stored in both upper and lower replay buffers, and mini-batches of experiences are sampled. Target Q-values are computed, and the upper and lower Q-networks are updated using gradient descent. The target networks are updated with the weights of the corresponding Q-networks every T episode. The algorithm proceeds until all episodes are completed.

V. NUMERICAL RESULTS AND DISCUSSIONS

In this section, we discussed the impact of evaporation coefficients and the number of pre-calculated paths on routing success rate and delay. Moreover, we compared the performance of our proposed routing algorithm with heuristic ant colony routing, centralized and distributed shortest path algorithms, and centralized deep reinforcement learning routing algorithm based on DQN in terms of success rate and delay on three different scales of Walker low Earth orbit satellite constellations.

A. Simulation Settings

We simulated the Walker constellation using the satellite toolbox in Matlab 2021 b. A topological snapshot was taken of a single operational period. The resulting snapshot was

Algorithm 3 Stigmergy Multi-Agent Hierarchical Deep Reinforcement Learning Routing Algorithm

Require: Environment Z , number of upper agents a^U , number of lower agents a^L , number of training episodes M , exploration policy π^e , discount factor γ , target network update frequency T

Ensure: Trained Q-networks $Q^{1,U}, \dots, Q^{a,U}$ and $Q^{1,L}, \dots, Q^{a,L}$

- 1: Initialize upper Q-networks $Q^{1,U}, \dots, Q^{a,U}$ and lower Q-networks $Q^{1,L}, \dots, Q^{a,L}$. Initialize target upper Q-networks $Q'^{1,U}, \dots, Q'^{a,U}$ and target lower Q-networks $Q'^{1,L}, \dots, Q'^{a,L}$ with the same weights as Q-networks.
- 2: **for** $m = 1, \dots, M$ **do**
- 3: Reset environment Z and get initial states $z^{1,U}, \dots, z^{a,U}$ and $z^{1,L}, \dots, z^{a,L}$
- 4: **for** each upper agent a^U **do**
- 5: Update pheromone by Algorithm 2
- 6: Select action $u^{a,U}$ using ϵ -greedy strategy based on upper Q-network $Q^{a,U}$
- 7: Execute action $u^{a,U}$ and give subgoal to lower agents
- 8: **for** each lower agent a^L **do**
- 9: Message Passing by Algorithm 1
- 10: Select action $u^{a,L}$ using ϵ -greedy strategy based on lower Q-network $Q^{a,L}$
- 11: Execute action $u^{a,L}$ and observe reward hiding r^{hid} and next state $z^{a,L'}$
- 12: Store experience $(z^{a,L}, u^{a,L}, r^{hid}, z^{a,L'})$ in lower replay buffer $D^{a,L}$
- 13: Sample mini-batch of experiences from $D^{a,L}$
- 14: Compute target lower Q-values $y^{a,L} = r^{hid} + \gamma \max_{u^{a,L'}} Q^{a,L'}(z^{a,L'}, u^{a,L'} | z^{a,h'}, u^{a,U'})$
- 15: Update lower Q-network $Q^{a,L}$ using gradient descent on $(y^{a,L} - Q^{a,L}(z^{a,L}, u^{a,L} | z^{a,U}, u^{a,U}))^2$
- 16: **if** $m \bmod T = 0$ **then**
- 17: Update target lower Q-network $Q'^{a,L}$ with weights of lower Q-network $Q^{a,L}$
- 18: **end if**
- 19: **end for**
- 20: Observe upper reward $r^{a,U}$ and next state $z'^{a,U}$
- 21: Store experience $(z^{a,U}, u^{a,U}, r^{a,U}, z'^{a,U})$ in upper replay buffer $D^{a,U}$
- 22: Sample mini-batch of experiences from $D^{a,U}$
- 23: Compute target upper Q-values $y^{a,U} = r^{a,U} + \gamma \max_{z^{a,U'}} Q^{a,U'}(z^{a,U'}, u^{a,U'})$
- 24: Update upper Q-network $Q^{a,U}$ using gradient descent on $(y^{a,U} - Q^{a,U}(z^{a,U}, u^{a,U}))^2$
- 25: **if** $m \bmod T = 0$ **then**
- 26: Update target upper Q-network $Q'^{a,U}$ with weights of upper Q-network $Q^{a,U}$
- 27: **end if**
- 28: **end for**
- 29: **end for**

imported into Python 3.8 using the Networkx library. The experiments were conducted on a device with an Intel i7

TABLE I
DETAILED SYSTEM PARAMETERS

Parameters	Value
Total Bandwidth	2000 Mbps
Bandwidth Demand	[10,40] Mbps
Request Arrival Rate	{10,15,20}
Request Duration	[6,10] s
Delay tolerance	[200,280] ms
Doppler shift threshold	$3.22 \times 10^{-11} m$
Nodes handled by each controller	16
Upper agent's learning rate	0.01
Upper agent's discount factor	0.9
Upper agent's batch size	32
Lower agent's learning rate	0.0001
Lower agent's discount factor	0.995
Lower agent's batch size	32
Lower agent's message passing times	4
Lower agent's dropout rate	0.01
Lower agent's readout units	35

13700k CPU, an RTX4080 GPU, and 32GB of RAM. PyTorch version 1.12.0 was utilized for the experiments.

In this article, we implement the proposed routing algorithm on three different configurations of Walker constellations, which are widely used in LEO satellite applications [32] to test the scalability of our algorithm. We deploy three satellite constellations with 92, 144, and 192 satellites, respectively. The number of orbital planes for each constellation is 8, 12, and 16, with 12 satellites per orbit.

Our detailed system parameters as shown in table I. We set the link bandwidth to 2000 Mbps and the Doppler frequency shift threshold to $3.22 \times 10^{-11} m$. The service traffic demand ranges from 10-40 Mbps, with the arrival rate following a Poisson distribution and intensity lambda values of 10, 15, and 20. The service duration is between 6-10 seconds, and the delay tolerance is between 200 and 280 ms. In terms of agent parameters, we configured the learning rate, discount factor, and batch size for both the upper and lower agents. Furthermore, for the graph neural network of the lower agent, we set the number of message-passing iterations to 4, the dropout rate to 0.01, and the number of readout units to 35.

B. Convergence Analysis

We evaluated the training convergence of both lower and upper agents on three types of walker satellite constellations. Fig. 5 and 6 show the convergence curves for the reward sum and reward variance of all agents. As shown in Fig. 5(a), the reward sum for all lower agents improved and converged with training in the 96-Walker constellation. During the initial training phase, the agents' reward sum was relatively low, at approximately 130. By the 600th episode, the reward sum gradually increased to around 350 and converged. As the scale of the Walker constellation expands, the reward sum for all agents increases due to the higher number of agents and identical reward rules for each agent.

As shown in Fig. 5(b), the reward variance for lower agents decreases with training episodes in all constellation scales. Furthermore, as the constellation scale grows larger, the reward variance for lower agents becomes smaller.

The reward sum for upper agents converges in all three Walker constellation scales. As shown in Fig. 6(a) the convergence trend for upper agents is not affected by the expansion of the constellation scale. Upper agents exhibit favorable convergence across all three Walker constellation scales. Additionally, Fig. 6(b) displays the variance of independent rewards among agents, where the reward variance between different agents gradually decreases with training episodes. It can be observed that the rewards for all upper agents also converge, albeit with some fluctuations due to the instability of rewards caused by pheromone iteration.

The 192 walker constellation initially achieved a high reward sum (over 500) during training, but with significant fluctuations. After 400 episodes, the reward sum slightly increased and fluctuations decreased, indicating convergence. This is due to the unique characteristics of the Walker constellation, where links near the poles are prone to failure. Lower agents managing these areas need to be trained to avoid these failed links. As the constellation scale increases, each agent manages fewer failed links, resulting in fewer penalties and potentially larger rewards early in training. This also explains the smaller reward sum variance for lower agents in larger constellations. This phenomenon does not appear among the upper agents, as they are not responsible for avoiding failed links within their domain. Therefore, as the scale of the constellation increases, the training curve of the upper agents remains unchanged.

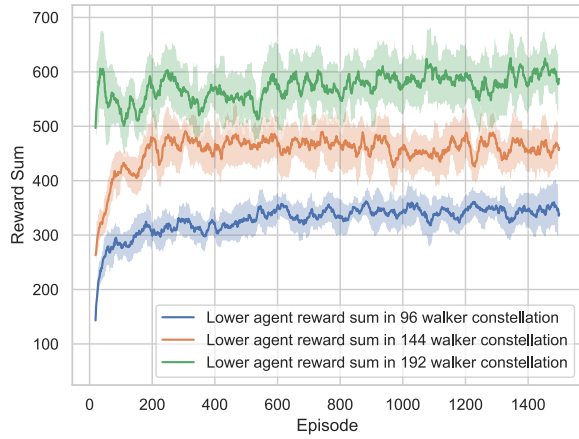
Therefore, we can conclude that during the training iterations, the rewards of all agents converged and the variance between agents also converged to a minimum value. This indicates that all agents achieved cooperation and learned effectively for routing optimization.

C. Parameter Comparison

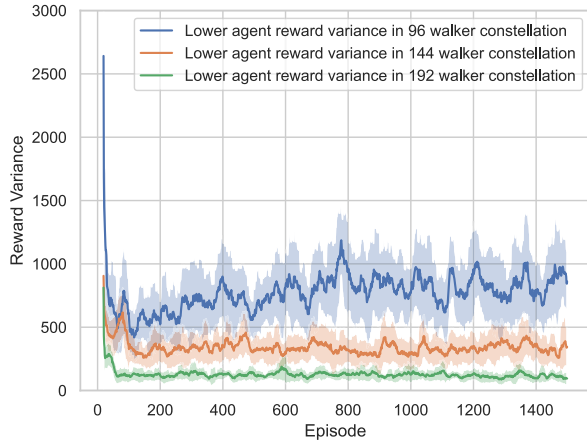
The convergence analysis demonstrates that our proposed algorithm achieves reward convergence. However, other factors also determine the algorithm's performance, such as the number of alternative paths for lower agents and pheromone accumulation for upper agents. We investigated the impact of different alternative path numbers ($k = 2, 3, 4, 5, 6$) and pheromone evaporation coefficients ($\rho = 0, 0.2, 0.4, 0.6, 0.8, 1$) on communication success rate and delay in the 144 walker constellation.

Figure 7(a) illustrates that the communication success rate escalates with an increasing number of alternative paths (k). The most notable improvement in the success rate occurs when ' k ' increases from 2 to 4, marking a 23.8% enhancement. However, the rate of this improvement slows down, as indicated by an 8.5% increase, when ' k ' transitions from 4 to 6. The pheromone evaporation coefficient (ρ) also impacts the success rate. The lowest success rate is recorded when $\rho = 0$, which suggests that an evaporation coefficient is necessary to prevent an excessive accumulation of pheromones. The highest communication success rate is observed when $k = 4$ and $\rho = 0.6$, marking an 8% increase compared to when $\rho = 0$.

Figure 7(b) reveals that the number of alternative paths and the evaporation coefficient have some impact on delay. More alternative paths result in greater communication delays,



(a) Reward sum for all lower agents.



(b) Reward variance between all lower agents.

Fig. 5. The rewards for lower agents were based on the performance during 1500 episodes of training for the Walker constellation with 96, 144, and 192 satellites.

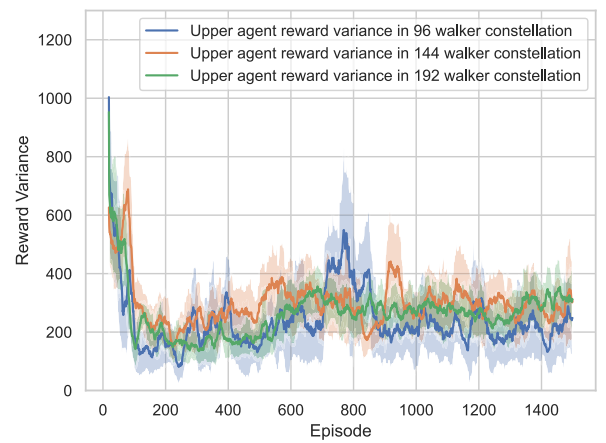
with an average delay increase of 13.25% from $k = 2$ to $k = 6$. This is because agents need to choose longer paths when some shortest path links fail. The increase in alternative paths provides more backup solutions but also brings greater delays. The impact of the evaporation coefficient (ρ) on communication delay is relatively small, with the largest delay occurring at $k = 4$ and $\rho = 0.4$, representing a 2.17% increase compared to $\rho = 0$.

D. Performance Analysis

We further tested the algorithm through network performance simulations, comparing several centralized and distributed routing optimization algorithms. These included the centralized shortest path routing algorithm (SR-C), the distributed shortest path routing algorithm (SR-D), the Ant Colony Routing Optimization algorithm (ACO) [33], and the centralized DQN routing optimization algorithm (DQN). For the ACO algorithm, we set the probability of greedy forwarding to 0.9, the pheromone evaporation ratio to 0.3, and the number of ant colony iterations to 50. The parameters for the DQN algorithm were kept consistent with the upper agent in our proposed algorithm. We compared their performance



(a) Reward sum for all upper agents.



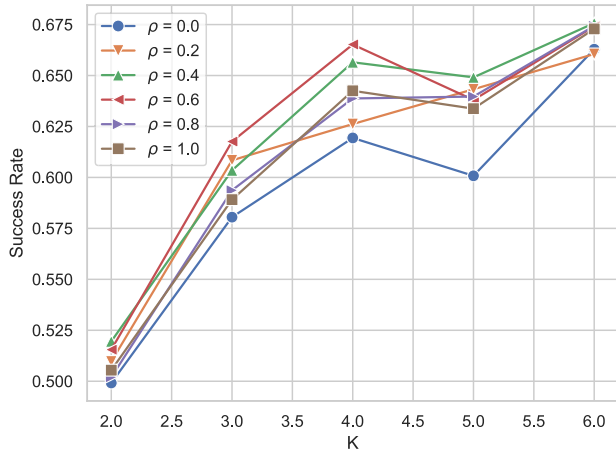
(b) Reward variance for all upper agents.

Fig. 6. The rewards for upper agents were based on the performance during 1500 episodes of training for the Walker constellation with 96, 144, and 192 satellites.

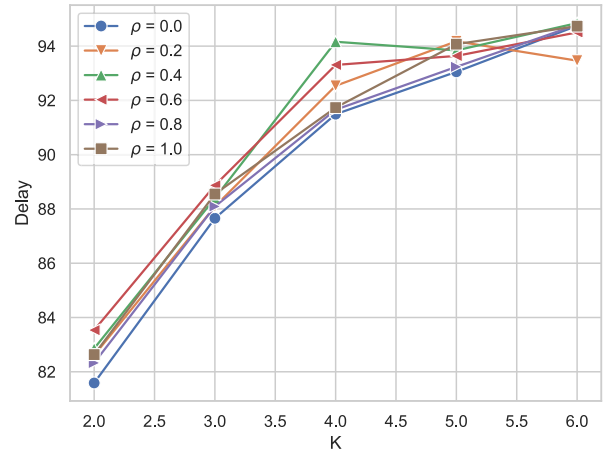
in terms of communication success rate, delay, and link utilization in the three Walker constellations, respectively.

1) *Communication Success Rate Performance Analysis:* A routing request is considered successful if all links in the routing path meet the constraints C1 to C4. Therefore, the communication success rate can be defined as the ratio of success routing request to all routing requests. Fig. 8 shows the communication success rate performance of five routing algorithms in the 96, 144, and 192 Walker constellations over 150-time steps.

In Fig. 8(a), our proposed algorithm achieves a noticeable improvement in the initial success rate, reaching over 0.77 when resources are sufficient. The success rate differences between the three comparison algorithms, SR-C, ACO, and SR-D, are not significant, ranging from 0.62 to 0.66. Our proposed algorithm's success rate increases by 18.46% compared to others. The DQN routing algorithm's success rate remains relatively stable, indicating it has not effectively converged in the 96 Walker constellation. As the time step increases, the system enters a high load state, and although our proposed algorithm maintains its lead, the gap between the success rates narrows (about 6% to 18%).

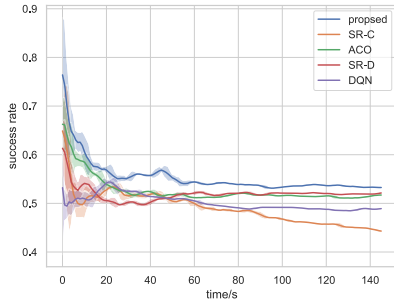


(a) The communication success rate for alternative paths $k=2, 3, 4, 5, 6$, and pheromone evaporation coefficients $\rho=0, 0.2, 0.4, 0.6, 0.8, 1$ In the 144 Walker constellation.

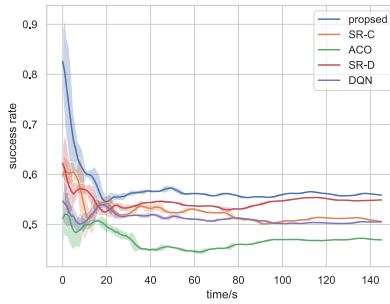


(b) The communication delay for alternative paths $k=2, 3, 4, 5, 6$, and pheromone evaporation coefficients $\rho=0, 0.2, 0.4, 0.6, 0.8, 1$ In the 144 Walker constellation.

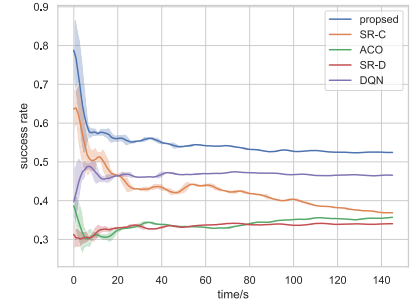
Fig. 7. The impact of alternative paths and pheromone evaporation coefficients on communication success rate and delay.



(a) The communication success rate in 96 walker constellations.



(b) The communication success rate in 144 walker constellation.



(c) The communication success rate in 192 walker constellation.

Fig. 8. The communication success rate in different walker constellations.

Fig. 8(b) and Fig. 8(c) demonstrate our proposed algorithm's significant advantage in communication success rate in larger-scale Walker constellations. In the 144-Walker constellation, our algorithm's highest average success rate reaches 0.83, with a 33.9% increase compared to centralized and distributed algorithms SR-C and SR-D. DQN and ACO have a communication success rate of less than 0.55, with ACO's performance declining significantly in larger-scale constellations. In the 192-Walker constellation, our proposed algorithm's communication success rate further improves, maintaining a relatively good rate even under high network load conditions, with a performance increase of up to 45.6% compared to benchmark algorithms.

These results demonstrate that our proposed algorithm is more adaptive and effective in larger-scale Walker constellations, successfully managing and optimizing routing amid increasing complexity and resource constraints.

2) *Communication Delay Performance Analysis:* Fig. 9 displays the communication delay performance of the five routing algorithms in 96, 144, and 192 Walker constellations over 150-time steps.

Constellation size significantly influences the performance of these routing algorithms. In the 96-Walker constellation, as shown in Fig. 9(a), our proposed algorithm exhibits excellent delay performance. The results of SR-C, SR-D,

and our proposed algorithm are comparable, slightly above 60ms, indicating that our algorithm can maintain relatively low delays when handling complex constellation network topologies, similar to other high-performance algorithms. However, the delay performance of the centralized routing algorithms based on DQN and ACO is less satisfactory due to challenges in finding optimal solutions within short iteration times in large-scale network topologies.

In both the 144 and 192 Walker constellations, the delay performance of the five algorithms displays a hierarchical pattern. As shown in Fig. 9(b), ACO's delay performance is the worst, consistently exceeding 90ms. The delay of our proposed algorithm and DQN is slightly higher than that of the SR-C algorithm. The SR-D algorithm, however, has the lowest delay, although it cannot identify faulty paths, leading to a higher failure rate of inter-domain routing. As shown in Fig. 9(c), the ant colony algorithm's delay remains above 90ms due to difficulties in finding feasible solutions in larger-scale networks. The performance of our proposed algorithm and the SR-C algorithm remains consistent, but the SR-D algorithm's delay is considerably lower than the other four algorithms, as it faces challenges in inter-domain routing transmission.

3) *Link Utilization Performance Analysis:* Link utilization refers to the ratio between the data traffic carried on a

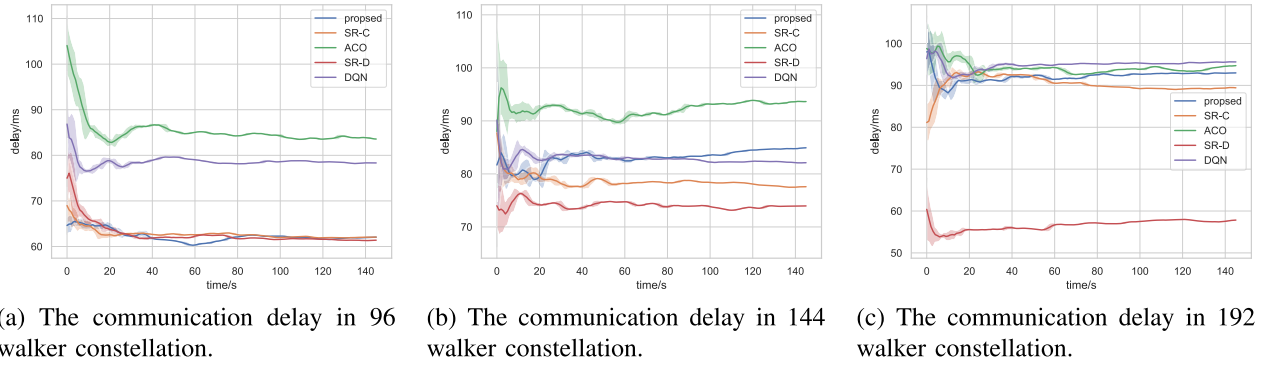


Fig. 9. The communication delay in different walker constellations.

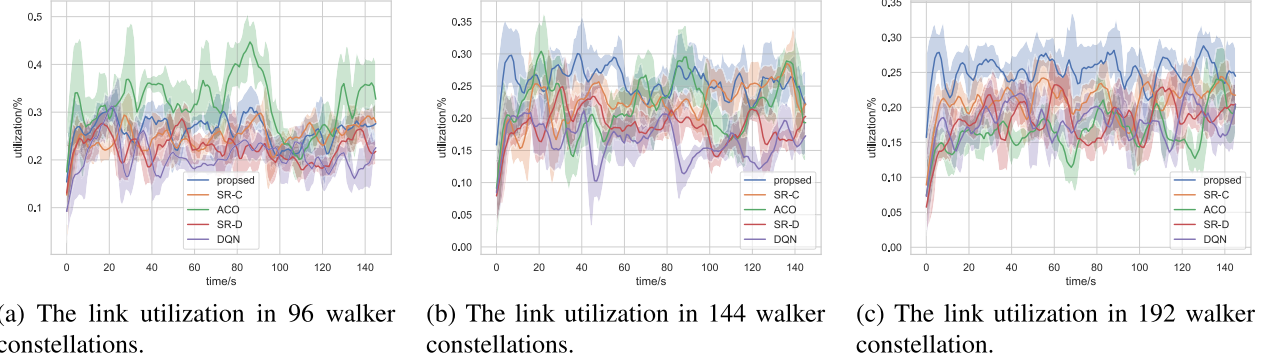


Fig. 10. The link utilization in different walker constellations.

network link and its maximum carrying capacity. As shown in Figure (10), it presents the link utilization performance of the five routing algorithms in the 96, 144, and 192 Walker constellations in 150 time steps.

As shown in Fig. 10(a), in the 96-walker constellation, the link utilization of the ACO algorithm is the highest, significantly exceeding the other three routing algorithms. At the 83-time step, its link utilization reaches as high as 0.43. Despite this, the ACO did not show any advantage in terms of communication success rate and delay. On the contrary, it had a much higher delay than the other comparison algorithms. This suggests that the ACO requires more routing hops to forward the same service requests, leading to it occupying more bandwidth resources. On the other hand, the algorithm we propose maintains good latency performance and, at a high communication success rate, sustains a relatively high link resource utilization, demonstrating a better load-balancing performance. In the 144 walker and 192 walker constellations, the algorithm we propose exhibits a higher link resource utilization performance. While maintaining a good communication success rate and latency performance, our proposed algorithm possesses the highest link resource utilization. This indicates that the algorithm we propose can utilize network resources more efficiently and achieve load balancing.

E. The Complexity Analysis of Algorithms

In this section, we will discuss the time complexity of our proposed algorithm and comparison algorithms, including the SR algorithm, and ACO algorithm. The time complexity of a trained DRL algorithm mainly depends on the process

of inputting states and outputting actions, which is directly related to the structure and number of parameters of the neural network.

1) Proposed Algorithm: The upper agent in our proposed algorithm uses a DQN algorithm, with time complexity $O(|E^{d,d'}|^2 \cdot f^U \cdot |u^{a,U}|)$, where $|E^{d,d'}|$ represents the number of cross-domain links and f^U is the feature dimension of each link. The lower agent uses a GNN+DQN algorithm. Ignoring node feature message passing, and considering only link feature message passing, the complexity is $O(|E^d|^2 \cdot f^L \cdot |u^{a,L}| \cdot T)$, where $|E^d|$ is the number of edges, f^L is the feature dimension of each edge, and T is the number of message passing times. Given that the routing decisions of the upper and lower agents in our algorithm are executed sequentially, the overall time complexity can be expressed as $O(|E^{d,d'}|^2 \cdot f^U \cdot |u^{a,U}| + |E^d|^2 \cdot f^L \cdot |u^{a,L}| \cdot T)$.

2) SR Algorithm: The SR algorithm is based on Dijkstra's algorithm, in which time complexity is $O((|V| + |E|) \log |V|)$, where $|V|$ is the number of all satellite nodes and $|E|$ is the number of all links between satellite nodes.

3) ACO Algorithm: The ACO algorithm has a per-iteration time complexity of $O(m|V| + |V|^2)$, where m is the number of ants and $|V|$ is the number of satellite nodes. Thus, for k iterations, the total time complexity is $O(k \cdot (m|V| + |V|^2))$.

Comparative Analysis: Our algorithm's time complexity is proportional to the square of the number of cross-domain (upper agent) and intra-domain (lower agent) links, with no clear advantages or disadvantages over comparison algorithms. However, due to the $|E_{i,j}^d| \ll |E|$ and $|E_{i,j}^{d,d'}| \ll |E|$

or $|V|$, making our algorithm more time-efficient in practice. Thus, despite domain sequential dependencies, our algorithm excels in scalability when managing expansive, distributed networks. Additionally, with the advancement of parallel computation for neural networks, AI algorithms can achieve reduced time complexity and potentially faster computation speeds, demonstrating their superior advantage over traditional algorithms.

VI. CONCLUSION

In this paper, we discuss the stigmergy multi-agent hierarchical deep reinforcement learning routing algorithm based on distributed SDN for satellite networks. To address the scalability challenges of routing algorithms and the difficulty in real-time global information gathering in large-scale satellite networks, we propose a deep reinforcement learning mechanism combining stigmergy and hierarchy. We introduce a pheromone-based stigmergy mechanism that enables different agents to collaborate during independent training and adopt hierarchical control to specifically address cross-domain routing. Simulation results demonstrate that our proposed algorithm exhibits good scalability in large-scale satellite networks.

REFERENCES

- [1] O. Kodheli et al., "Satellite communications in the new space era: A survey and future challenges," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 70–109, 1st Quart., 2021.
- [2] Z. Wang et al., "Learning to routing in UAV swarm network: A multi-agent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6611–6624, May 2023.
- [3] W. He, H. Yao, T. Mai, F. Wang, and M. Guizani, "Three-stage Stackelberg game enabled clustered federated learning in heterogeneous UAV swarms," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 9366–9380, Jul. 2023, doi: [10.1109/TVT.2023.3246636](https://doi.org/10.1109/TVT.2023.3246636).
- [4] T. Mai, H. Yao, N. Zhang, W. He, D. Guo, and M. Guizani, "Transfer reinforcement learning aided distributed network slicing optimization in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 6, pp. 4308–4316, Jun. 2022.
- [5] F. Chai, Q. Zhang, H. Yao, X. Xin, R. Gao, and M. Guizani, "Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite IoT," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7783–7795, Jun. 2023, doi: [10.1109/TVT.2023.3238771](https://doi.org/10.1109/TVT.2023.3238771).
- [6] J. C. McDowell, "The low earth orbit satellite population and impacts of the SpaceX starlink constellation," *Astrophys. J. Lett.*, vol. 892, no. 2, p. L36, Apr. 2020.
- [7] T. Darwish, G. Kurt, H. Yanikomeroglu, G. Lamontagne, and M. Bellemare, "Location management in IP-based future LEO satellite networks: A review," 2021, *arXiv:2101.08336*.
- [8] J. Wang, L. Li, and M. Zhou, "Topological dynamics characterization for LEO satellite networks," *Comput. Netw.*, vol. 51, no. 1, pp. 43–53, Jan. 2007.
- [9] A. Papa, T. De Cola, P. Vizarreta, M. He, C. M. Machuca, and W. Kellerer, "Dynamic SDN controller placement in a LEO constellation satellite network," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2018, pp. 206–212.
- [10] Y. Li et al., "An elastic resource allocation algorithm based on dispersion degree for hybrid requests in satellite optical networks," *IEEE Internet Things J.*, vol. 9, no. 9, pp. 6536–6549, May 2022.
- [11] E. Ekici, I. F. Akyildiz, and M. D. Bender, "A distributed routing algorithm for datagram traffic in LEO satellite networks," *IEEE/ACM Trans. Netw.*, vol. 9, no. 2, pp. 137–147, Apr. 2001.
- [12] Q. Chen, X. Chen, L. Yang, S. Wu, and X. Tao, "A distributed congestion avoidance routing algorithm in mega-constellation network with multi-gateway," *Acta Astronautica*, vol. 162, pp. 376–387, Sep. 2019.
- [13] H. K. Ravuri, M. T. Vega, J. van der Hooft, T. Wauters, B. Da, and F. De Turck, "On routing scalability in flat SDN architectures," in *Proc. 11th Int. Conf. Netw. Future (NoF)*, Oct. 2020, pp. 23–27.
- [14] T. Pan, T. Huang, X. Li, Y. Chen, W. Xue, and Y. Liu, "OPSPF: Orbit prediction shortest path first routing for resilient LEO satellite networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2019, pp. 1–6.
- [15] P. Kumar, S. Bhushan, D. Halder, and A. M. Baswade, "FybrLink: Efficient QoS-aware routing in SDN enabled future satellite networks," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 3, pp. 2107–2118, Sep. 2022.
- [16] W. Zhou et al., "Fast convergence RWA algorithm based on ant colony algorithm," in *Proc. IEEE Comput., Commun. IoT Appl. (ComComAp)*, Dec. 2020, pp. 1–3.
- [17] Y. Dong, S. Zhao, H. da Ran, Y. Li, and Z. Zhu, "Routing and wavelength assignment in a satellite optical network based on ant colony optimization with the small window strategy," *J. Opt. Commun. Netw.*, vol. 7, no. 10, pp. 995–1000, Oct. 2015.
- [18] Y. Zhao, H. Yao, Z. Qin, and T. Mai, "Collaborate Q-learning aided load balance in satellites communications," in *Proc. Int. Wireless Commun. Mobile Comput. (IWCMC)*, Jun. 2021, pp. 968–973.
- [19] F. Zheng, C. Wang, Z. Zhou, Z. Pi, and D. Huang, "LEO laser microwave hybrid inter-satellite routing strategy based on modified Q-routing algorithm," *EURASIP J. Wireless Commun. Netw.*, vol. 2022, no. 1, pp. 1–18, Dec. 2022.
- [20] H. Yao, T. Mai, C. Jiang, L. Kuang, and S. Guo, "AI routers & network mind: A hybrid machine learning paradigm for packet routing," *IEEE Comput. Intell. Mag.*, vol. 14, no. 4, pp. 21–30, Nov. 2019.
- [21] M. Dorigo, E. Bonabeau, and G. Theraulaz, "Ant algorithms and stigmergy," *Future Gener. Comput. Syst.*, vol. 16, no. 8, pp. 851–871, Jun. 2000.
- [22] X. Xu, R. Li, Z. Zhao, and H. Zhang, "Stigmergic independent reinforcement learning for multiagent collaboration," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 4285–4299, Sep. 2022.
- [23] V. Bolshakov, A. Alfimtsev, S. Sakulin, and N. Bykov, "Deep reinforcement ant colony optimization for swarm learning," in *Advances in Neural Computation, Machine Learning, and Cognitive Research V*. Cham, Switzerland: Springer, 2021, pp. 9–15.
- [24] S. Pateria, B. Subagdja, A.-H. Tan, and C. Quek, "Hierarchical reinforcement learning: A comprehensive survey," *ACM Comput. Surv.*, vol. 54, no. 5, pp. 1–35, 2021.
- [25] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, nos. 1–2, pp. 181–211, Aug. 1999.
- [26] Y. Shahar, "A framework for knowledge-based temporal abstraction," *Artif. Intell.*, vol. 90, nos. 1–2, pp. 79–133, Feb. 1997.
- [27] Z. Han, C. Xu, Z. Xiong, G. Zhao, and S. Yu, "On-demand dynamic controller placement in software defined satellite-terrestrial networking," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 3, pp. 2915–2928, Sep. 2021.
- [28] L. Chen, F. Tang, and X. Li, "Mobility- and load-adaptive controller placement and assignment in LEO satellite networks," in *Proc. IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10.
- [29] Q. Yang, L. Tan, and J. Ma, "Doppler characterization of laser inter-satellite links for optical LEO satellite constellations," *Opt. Commun.*, vol. 282, no. 17, pp. 3547–3552, Sep. 2009.
- [30] F. A. Oliehoek, "Decentralized POMDPs," in *Reinforcement Learning (Adaptation, Learning, and Optimization)*, vol. 12, M. Wiering and M. van Otterlo, Eds. Berlin, Germany: Springer, 2012, doi: [10.1007/978-3-642-27645-3_15](https://doi.org/10.1007/978-3-642-27645-3_15).
- [31] P. Almasan, J. Suárez-Varela, K. Rusek, P. Barlet-Ros, and A. Cabellos-Aparicio, "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *Comput. Commun.*, vol. 196, pp. 184–194, Dec. 2022.
- [32] A. Turner, "Constellation design using Walker patterns," in *Proc. AIAA/AAS Astrodyn. Spec. Conf. Exhib.*, Aug. 2002, p. 4636.
- [33] T. Stützle, "ACO algorithms for the travelling salesman problem," *Evol. Algorithm Eng. Comput. Sci.*, vol. 4, pp. 163–183, Jan. 1999.



Yuanfeng Li is currently pursuing the Ph.D. degree in optical engineering with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China. His research interests include optical satellite network architecture, network artificial intelligence, multiagent systems, space-terrestrial integrated networks, and network resource allocation.



Qi Zhang (Member, IEEE) received the Ph.D. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2005. She is currently a Professor with the School of Electric Engineering, BUPT. Her research interests include optical access networks, optical fiber communication, and satellite communication. Within this area, she has authored or coauthored over 100 SCI articles.



Xiangjun Xin was born in Hebei, China, in 1969. He received the Ph.D. degree from the School of Electric Engineering, Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2004. He is currently a Professor with the School of Electric Engineering, BUPT. He is a member of the State Key Laboratory of Information Photonics and Optical Communications, BUPT. His main research interests include broadband optical transmission technologies, optical sensor, all-optical networks, and satellite communication. Within this area, he has authored or coauthored over 100 SCI articles.



Haipeng Yao (Senior Member, IEEE) received the Ph.D. degree from the Department of Telecommunication Engineering, University of Beijing University of Posts and Telecommunications (BUPT), in 2011. He is currently a Professor with BUPT. His research interests include future network architecture, network artificial intelligence, networking, space-terrestrial integrated networks, network resource allocation, and dedicated networks. He has published more than 150 papers in prestigious peer-reviewed journals and conferences. He has

served as an Associate Editor for IEEE TRANSACTIONS ON MOBILE COMPUTING and IEEE TRANSACTIONS ON SUSTAINABLE COMPUTING. He has also served as a member of the technical program committee and the Symposium Chair for a number of international conferences, including the IWCMC 2019 Symposium Chair and the ACM TUR-C SIGSAC2020 Publication Chair.



Ran Gao received the Ph.D. degree in electronic science and technology from the Beijing Institute of Technology, Beijing, China, in 2015. He is currently an Associate Professor with the Advanced Research Institute of Multidisciplinary Science, Beijing Institute of Technology. His research interests include fiber optical sensors, optical communication, and measurement instruments.



F. Richard Yu (Fellow, IEEE) received the Ph.D. degree in electrical engineering from The University of British Columbia (UBC) in 2003. His research interests include connected/autonomous vehicles, artificial intelligence, cyber security, and wireless systems. He has been named in the Clarivate Analytics list of Highly Cited Researchers since 2019. He is an Elected Member of the Board of Governors of the IEEE VTS. He is a fellow of the Canadian Academy of Engineering (CAE), Engineering Institute of Canada (EIC), and IET. He received several best paper awards from some first-tier conferences. He is the Editor-in-Chief of the IEEE VTS Mobile World Newsletter. He is a Distinguished Lecturer of the IEEE in VTS and ComSoc.